

Week5 - Transformer

1. Before Transformer

2. Attention

3. Transformer

Transformer 모델 recap

4. GPT, BERT, BART

GPT

BERT

BART = GPT + BERT (Bidirectional Auto-Regressive Transformer)

1. Before Transformer

1. RNN

- 순차적으로 시퀀스 데이터를 처리 (문장, 음성 등)
- 이전 타임스텝 정보를 hidden state로 다음에 전달 → 문맥 반영 가능
- 문제: 기울기 소실/폭발 → 장기 의존성 학습 어려움

2. LSTM

- cell state와 3가지의 gate → forget, input, output gate로 장기 의존성 문제 완화
- 중요한 정보 유지, 불필요 정보는 삭제
- 연산량이 많아 느림

3. 경량화된 GRU 등장

- LSTM보다 단순 (Reset Gate, Update Gate만 사용)
- 연산량 감소, 성능은 비슷
- cell state 대신 hidden state 1개만 사용

4. Seq2Seq

- encoder-decoder 구조

- RNN기반의 LSTM, GRU 등으로 cell이 구현이 되어있는 구조
- 입력 시퀀스를 하나의 context vector로 압축하여 변환하게됨 → 하나의 고정된 벡터로 압축하다보니 정보가 손실되는 병목 현상 발생한다는 문제점이 발생
- attention 메커니즘을 도입해보자 ⇒ 인코더의 모든 시점을 디코더가 참고할 수 있도록 하자!

4. Seq2Seq + attention

- Encoder의 모든 hidden state를 저장 후, Decoder가 단어 생성 시 참조
- 특정 시점에서 중요한 단어에 더 많은 가중치 부여

2. Attention

- **Query:** 현재 처리하려는 벡터
- **Key:** Query와의 유사도 비교 대상
- **Value:** 유사도 가중치가 적용되는 값
- **Dot-Product Attention:**
 1. Query-Key 내적 → Attention Score
 2. Softmax → 가중치
 3. Value에 가중치 곱 → 합산
 4. Decoder hidden state와 연결
- 장점: 문장 전체 문맥을 한 번에 고려 가능

3. Transformer

- seq2seq에서 RNN을 뺀 구조 ⇒ 즉 우리는 전적으로 오직 attention만 필요하다는 의미
- 크게 세 가지로 나눌 수 있음
 - input into encoder
 - encoder
 - decoder

1. input into encoder

- 각 입력 시퀀스를 학습에 사용되는 벡터로 임베딩 하는 임베딩 layer 구축
- positional encoding

2. Encoder

- **self-attention** ⇒ 자기 자신에게 attention을 적용하는것
- scaled dot-product attention
- **multi-head attention**

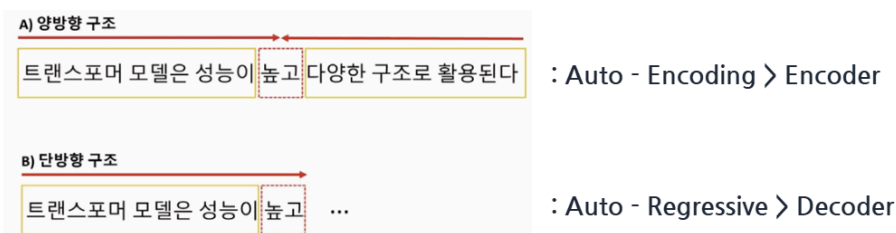
3. Decoder

- **masked multi-head attention** (미래 정보를 차단)
- encoder-decoder attention
- 공통: residual connection & layer normalization ⇒ 정규화 단계 (batch 정규화가 불가능하므로 layer 정규화를 진행한 것임)

Transformer 모델 recap

Transformer

: Auto - Encoding + Auto - Regressive



4. GPT, BERT, BART

모델	구조	학습 방식	방향성
GPT	Decoder Only	Auto-Regressive (다음 단어 예측)	Unidirectional
BERT	Encoder Only	Auto-Encoding (MLM, NSP)	Bidirectional

모델	구조	학습 방식	방향성
BART	Encoder+Decoder	Auto-Encoding + Auto-Regressive	Bidirectional + Unidirectional

GPT

1. unsupervised pre-training

- next word를 예측하도록 학습
- 모델이 장기적인 문맥을 이해할 수 있도록 함

2. supervised fine-tuning

- task-specific layer 추가
- 발전: GPT-1 → GPT-2(zero-shot) → GPT-3(few-shot) → GPT-3.5(RLHF) → GPT-4(multimodal) → GPT-5

BERT

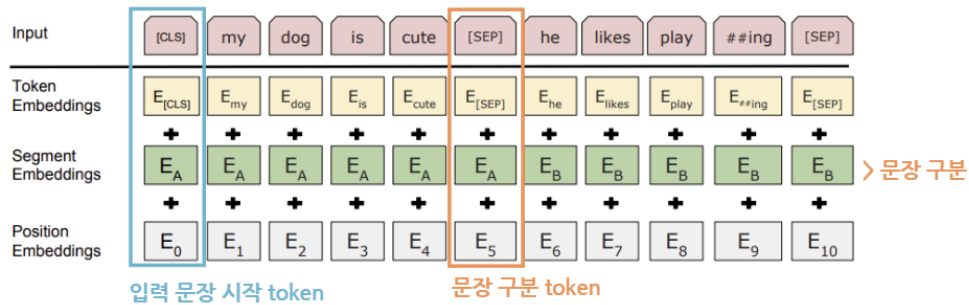
- transformer의 encoder만 사용해서 이걸 쌓아올려서 만든 모델
- semi-supervised learning
- **Masked Language Model (MLM)**: 15% 단어 마스킹, 문맥 기반 예측
- **Next Sentence Prediction (NSP)**: 문장 연결 관계 학습
- Bidirectional 문맥 학습에 강점

1. unsupervised pre-training ⇒ masked language model

- 일부만 masking을 진행 (15% 정도만 마스킹)
 - 모든 입력 토큰에 대해 학습

1) Unsupervised pre-training : Next Sentence Prediction

- 질의응답(QA) / 자연어 추론(NLI) task



- 50% : 실제로 A 다음에 오는 문장
- 50% : 코퍼스에서 무작위로 뽑은 문장

- 문장이 참인지 거짓인지는 맨 앞의 CLS token으로 판단 가능

2. supervised fine-tuning ⇒ end-to-end

- GPT와 다르게 pre-training과 fine-tuning에서 구조 차이가 없음
- token 수준으로 진행하는 task

BART = GPT + BERT (Bidirectional Auto-Regressive Transformer)

- 두 가지 모델을 차용한 모델
- Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and
- BART는 denoising autoencoder ⇒ 입력에 노이즈 추가 후 원래 문장을 복원하면서 문장의 다양한 요소를 학습하도록 함
 - text에 masking을 주는 것이 노이즈를 주는것
 - 자연어 생성, 번역, 이해 모두 가능!
- 차이점1
 - text에 masking을 추가하는 것 외에도 다양한 마스킹 기법을 사용한다!
 - token masking, token deletion(어느 위치에서 삭제되었는지도 맞춰야 함 ⇒ 중요하지 않은 정보 자동 필터링 가능), sentence permutation, document

rotation, text infilling, ...

- 차이점2
 - encoder의 마지막 계층과 decoder의 각 계층에서만 attention을 수행
 - 입력 문장 전체의 의미를 가장 잘 반영하는 벡터를 인코더에서 뽑아 attention을 수행하기 때문에 메모리 사용량을 줄일 수 있다는 장점이 존재